

Mystery_Solver: Design, Probabilistic Formulation, Implementation Analysis, and Critical Evaluation of a Bayesian Mystery-Solving Application

Group 8 Project Report

Abstract—This paper presents a complete technical report on **Mystery_Solver**, a compact artificial intelligence application that uses a Bayesian Network to estimate the most likely culprit in a fictional theft case. The application combines an interactive Streamlit front end with a probabilistic reasoning back end written in Python and implemented using **pgmpy**. Although the project is small in code size, it is pedagogically rich because it demonstrates probabilistic graphical models, conditional probability distributions, exact posterior inference through variable elimination, and human-interpretable decision support. This report reconstructs the software architecture from source inspection, formalizes the underlying probabilistic model, explains the execution flow from user input to posterior output, and evaluates the implementation from both an AI and software engineering perspective. The analysis shows that the project is conceptually sound as an educational demonstration, but it also contains practical weaknesses such as a non-portable copied virtual environment, incomplete use of evidence fields, limited documentation, and the absence of automated tests. In addition to describing the existing application, the report discusses reproducibility, representative inference scenarios, implementation limitations, and a roadmap for improving the project into a more complete academic artifact. The paper is written as a professional final report in conference-style format and includes equations, tables, figures, and explicit references to the technical sources relevant to the implementation.

Index Terms—Bayesian network, probabilistic inference, variable elimination, Streamlit, pgmpy, Python, educational AI software, software engineering analysis

I. INTRODUCTION

Artificial intelligence systems often operate in environments where information is uncertain, incomplete, noisy, or contradictory. In such cases, decision-making cannot depend only on deterministic rules. Instead, an intelligent system may need to combine partial evidence, estimate how likely different explanations are, and update beliefs as new observations become available. Bayesian Networks are one of the most influential frameworks developed for this purpose because they encode causal or dependency structure in a compact directed graph and attach probabilities to that structure in a mathematically consistent way [1]. Exact and approximate algorithms can then be used to answer probabilistic queries such as posterior belief updating, most-likely explanation, or causal reasoning [2].

The software artifact studied in this report, **Mystery_Solver**, is a small interactive project that uses these ideas in a narrative problem setting. Rather than modeling a medical diagnosis problem, a robot control task, or a real-world expert system, the program is framed as a

fictional investigation: a valuable manuscript has disappeared, three suspects are possible culprits, and several clues may support or weaken suspicion against each suspect. The user enters clues through a browser-based interface, and the application returns a posterior probability distribution over the suspects.

At first glance the project appears simple, but that simplicity makes it valuable for academic analysis. It provides a self-contained example of how an AI concept taught in class can be mapped into code, visualized in a user-facing interface, and critically assessed in terms of assumptions, quality, and reproducibility. The project also illustrates an important reality of student software: conceptual understanding and implementation quality do not always advance at the same pace. A project may encode the right mathematical idea while still suffering from documentation gaps, environment problems, or interface inconsistencies.

This report therefore has four goals. First, it explains what the project is and what problem it tries to solve. Second, it formally describes the probabilistic model and the system architecture. Third, it evaluates the implementation quality of the existing codebase. Fourth, it provides concrete recommendations for turning the project from a working prototype into a stronger academic submission. All software-specific statements in this paper were derived from direct inspection of the provided source files, and all background claims about the tools and methods are supported by standard references and official documentation [3]–[7].

II. BACKGROUND

A. Bayesian Networks as AI Models

A Bayesian Network is a directed acyclic graph in which each node denotes a random variable and each edge denotes a probabilistic dependency or influence relation [1]. The graph structure allows a joint distribution over many variables to be factored into local conditional distributions. This factorization is important because direct enumeration of a full joint table grows exponentially with the number of variables, while a structured graph can often represent the same knowledge much more compactly.

If $X_1, \dots, X_n$ are the variables in a Bayesian Network and $\text{Pa}(X_i)$ denotes the set of parents of X_i , the joint distribution

factorizes as

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i)). \quad (1)$$

This equation is the core reason Bayesian Networks are useful in AI. It lets the model designer express local assumptions such as “forced entry is more likely if the cat burglar is guilty” without directly specifying every possible global combination of clues.

B. Exact Inference and Variable Elimination

Once a Bayesian Network is defined, the main computational question is how to answer posterior queries such as $P(X \mid e)$, where e is observed evidence. One of the standard exact methods is variable elimination, which computes marginals by multiplying relevant factors and summing out non-query variables in a chosen order [2]. While the worst-case cost can still be exponential in graph width, the method is often practical for small or moderately sized networks and is especially appropriate for educational demonstrations.

For the present project, variable elimination is a sensible inference choice because the number of variables is small, every variable is discrete, and the application needs exact posteriors rather than approximate samples. The use of `pgmpy`’s `VariableElimination` engine is therefore technically appropriate [3], [4].

C. Supporting Software Frameworks

The project relies on several external tools:

- **Python** as the general-purpose language and runtime [7].
- **pgmpy** as the probabilistic graphical modeling library [3], [4].
- **Streamlit** as the interface framework for browser-based interaction [5].
- **Graphviz** for structural graph rendering [6].

These tools are well matched to a student AI project because they reduce boilerplate and let the main focus remain on model design and reasoning.

III. PROJECT OBJECTIVE AND SCOPE

The explicit objective of `Mystery_Solver` is to estimate the probability that each of three suspects is guilty after selected evidence is observed. The user can explore different evidence combinations and immediately see how those combinations change the posterior belief distribution. The core educational aim is therefore not only to produce a final answer, but to help the user understand *why* probabilities shift.

The fictional case involves a stolen manuscript and three suspects:

- **A: The Scholar**
- **B: The Butler**
- **C: The Cat Burglar**

The scope of the application is intentionally narrow. It does not use machine learning to fit parameters from data. It does not scrape evidence from external sources. It does not attempt natural-language processing or open-ended reasoning. Instead,

TABLE I
MAIN FILES AND THEIR ROLES

Path	Type	Role
<code>main.py</code>	Source file	Streamlit application entry point, UI logic, evidence assembly, result display
<code>supports/mystery_solver.py</code>	Source file	Bayesian Network definition, CPDs, model validation, Graphviz plot creation
<code>supports/__init__.py</code>	Package file	Marks supports as a Python package
<code>requirements.txt</code>	Metadata file	Dependency list; filename is non-standard
<code>venv/</code>	Environment folder	Shared virtual environment copied from another computer

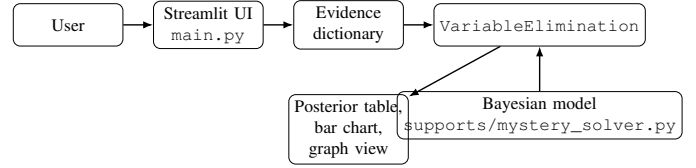


Fig. 1. High-level software architecture of the project.

the system operates over a small fixed set of variables, all of which are discrete and defined in advance. This makes the project suitable for demonstration and classroom presentation, but it also means the conclusions generated by the application are only as trustworthy as the hand-crafted probabilities encoded by the developer.

In academic terms, `Mystery_Solver` should be viewed as an *interpretable probabilistic reasoning prototype*. It sits somewhere between a classroom simulation and a lightweight expert-system demonstration. That framing is important because it clarifies how the project should be judged: conceptual correctness, clarity, and explainability matter more than industrial scalability.

IV. SOFTWARE ARTIFACT OVERVIEW

A. Directory Structure

The actual project is located in the inner directory `I:\Mystery_Solver\Mystery_Solver`. The important files are summarized in Table I.

B. Architectural Decomposition

The overall system architecture is shown in Fig. 1. From a software engineering viewpoint, the project has a reasonably clean two-layer structure:

- 1) A presentation layer implemented with Streamlit.
- 2) A model layer implemented with `pgmpy` and standard Python.

This separation is beneficial because it keeps the Bayesian model reusable even if the interface is replaced later.

C. Dependency Audit

The dependency list contains the following packages:

TABLE II
DEPENDENCY AUDIT

Package	Used?	Observation
pgmpy	Yes	Required for network representation and inference
streamlit	Yes	Required for the web interface
pandas	Yes	Used for probability table formatting and chart data
graphviz	Yes	Used for network visualization
numpy	Indirect	Likely dependency of scientific stack, not explicitly imported in code
networkx	No direct use	Possibly a leftover from experimentation
matplotlib	No direct use	Not explicitly used in the current app
ipykernel, jupyter	No direct use	Likely included for notebook-based development rather than application runtime

pgmpy, streamlit, networkx, matplotlib, numpy, pandas, graphviz, ipykernel, jupyter

Not all listed packages are directly used in the code. Table II separates essential dependencies from likely development leftovers.

This audit matters because academic projects are judged not only on whether they run, but also on how cleanly they are packaged. Listing unused packages is not fatal, but it does suggest that the environment was not fully curated before submission.

V. PROBABILISTIC MODEL FORMULATION

A. Variables and Graph Topology

The hidden cause variable is `GuiltyParty`, denoted by G , with three states:

$$G \in \{A, B, C\}. \quad (2)$$

The evidence variables are:

- ForcedEntry ($FE \in \{Yes, No\}$)
- AlibiA ($A_A \in \{Yes, No\}$)
- AlibiB ($A_B \in \{Yes, No\}$)
- AlibiC ($A_C \in \{Yes, No\}$)
- Fingerprints ($FP \in \{None, A, B, C\}$)
- SecurityFootage ($SF \in \{Yes, No\}$)

The graph contains a directed edge from G to each evidence variable. Fig. 2 shows the topology.

B. Joint Distribution

Because all evidence nodes have the same parent, the full joint distribution takes a particularly simple form:

$$P(G, FE, A_A, A_B, A_C, FP, SF) = P(G)P(FE | G)P(A_A | G)P(A_B | G)P(A_C | G)P(FP | G)P(SF | G). \quad (3)$$

This factorization implies conditional independence among the evidence variables given G . In words, once the culprit is

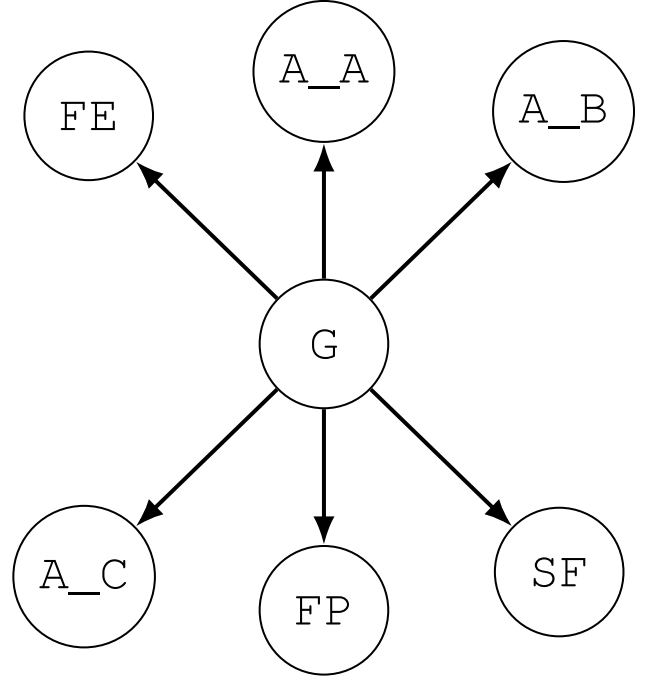


Fig. 2. Bayesian Network used by the application.

known, the model assumes the clue variables do not provide additional information about each other. For an educational demonstration this is a useful simplification because it keeps the graph legible and the posterior formula easy to explain. In a richer real-world model, some of these variables might have direct interdependencies.

C. Prior and Conditional Probability Tables

The prior over suspects is uniform:

$$P(G = A) = P(G = B) = P(G = C) = \frac{1}{3}. \quad (4)$$

The CPDs encode narrative assumptions. A subset of the most important assignments is summarized in Table III. For instance, the cat burglar is modeled as much more likely to use forced entry, while the scholar and butler are modeled as less likely to do so. Similarly, each suspect is less likely to have an alibi if that suspect is the true culprit.

D. Posterior Computation

Suppose a set of evidence values e is observed. The application computes the posterior probability of each suspect:

$$P(G | e) = \frac{P(G, e)}{P(e)}. \quad (5)$$

Because of the factorized graph, this becomes

$$P(G | e) = \alpha P(G) \prod_{i=1}^m P(E_i = e_i | G), \quad (6)$$

where α is the normalization constant required to make the probabilities sum to one.

TABLE III
REPRESENTATIVE CPD ENTRIES

Variable	State	Selected probabilities
ForcedEntry	Yes	$P(Yes A) = 0.1, P(Yes B) = 0.1, P(Yes C) = 0.9$
AlibiA	Yes	$P(Yes A) = 0.3, P(Yes B) = 0.8, P(Yes C) = 0.8$
AlibiB	Yes	$P(Yes A) = 0.8, P(Yes B) = 0.3, P(Yes C) = 0.7$
AlibiC	Yes	$P(Yes A) = 0.7, P(Yes B) = 0.7, P(Yes C) = 0.2$
Fingerprints	Match	Matching prints strongly favor the corresponding suspect
SecurityFootage	Yes	$P(Yes A) = 0.4, P(Yes B) = 0.4, P(Yes C) = 0.2$

Equation (6) explains why the application responds so clearly to strong evidence. If multiple clue likelihoods all strongly favor the same suspect, their product rapidly dominates the normalized posterior.

VI. IMPLEMENTATION WALKTHROUGH

A. Model Construction

The function `build_bayesian_network()` instantiates a `DiscreteBayesianNetwork` object, adds the graph edges, constructs each `TabularCPD`, attaches the CPDs to the model, and finally checks model validity using `model.check_model()`. This final validity check is one of the stronger engineering decisions in the code because it protects against malformed probability tables and graph inconsistencies.

B. Visualization

The helper function `create_graphviz_plot()` constructs a directed `Graphviz` object from the model nodes and edges. The front end then attempts to render this object using `Streamlit`. This is a useful user-facing feature because it gives the audience a structural explanation of the model rather than only a numerical output.

C. Front-End Logic

The `Streamlit` application begins by importing required modules, setting the page configuration, and attempting to build the model and inference engine. If initialization succeeds, the page exposes sidebar widgets representing evidence variables. Unknown values are represented internally as `None`, allowing the code to omit unspecified evidence from the final query.

The UI-to-model mapping is summarized in Table IV.

TABLE IV
MAPPING FROM INTERFACE CONTROLS TO MODEL VARIABLES

Sidebar prompt	Internal value	Model variable
Forced Entry?	Yes/No/None	ForcedEntry
Alibi for Scholar A?	Yes/No/None	AlibiA
Alibi for Butler B?	Yes/No/None	AlibiB
Alibi for Cat Burglar C?	Yes/No/None	AlibiC
Fingerprints Found?	None/A/B/C/None	Fingerprints
Useful Security Footage?	Yes/No/None	SecurityFootage

TABLE V
RUNTIME EXECUTION FLOW

Stage	Code responsibility	Purpose
Initialization	Build model and inference engine	Prepare valid Bayesian Network before user interaction
Input	Sidebar widgets	Collect clue values from the user
Filtering	Evidence dictionary creation	Remove unknown evidence from the query
Inference	<code>VariableElimination</code>	Compute posterior over suspects
Output	<code>DataFrame</code> and charts	Present probabilities in readable form

D. Execution Flow

The end-to-end execution flow of the application is deliberately simple, which is one reason the project is useful as an AI demonstration. When the `Streamlit` app is launched, the first major task is initialization. The application imports the model-building function from the support module, constructs the Bayesian Network, validates the CPDs, and creates an instance of `VariableElimination`. Only after this setup succeeds does the interface allow the user to interact with the mystery.

During interaction, the sidebar acts as the evidence collection panel. Each clue has a small set of discrete choices. For most binary clues, the options are *Yes*, *No*, and *Unknown*. The *Unknown* value is important because it prevents the program from forcing the user to invent evidence. Instead, unknown clues are excluded from the evidence dictionary. This design is consistent with Bayesian inference: the posterior should be conditioned only on evidence that has actually been observed.

Once the solve button is pressed, the app converts the selected clues into a dictionary whose keys match the Bayesian Network variable names. The inference engine then computes the posterior distribution over `GuiltyParty`. Finally, the posterior values are converted into a `pandas DataFrame` and displayed as a table and bar chart. This sequence is summarized in Table V.

E. Code Quality Observations

The implementation is readable and compact. The use of a separate support module for the Bayesian model is a positive design choice because it prevents the `Streamlit` file from becoming responsible for both interface rendering and mathematical model definition. The import error handling in

`main.py` is also helpful because it gives the user a visible message if the support module cannot be loaded.

However, the project would benefit from a clearer separation between domain configuration and application logic. At present, the CPDs are hard-coded directly inside `build_bayesian_network()`. This is acceptable for a course prototype, but it means changing the story assumptions requires editing source code rather than adjusting a configuration file. A future version could store suspect names, evidence options, and CPD values in a structured file such as JSON or YAML, then build the network from that configuration. Such a change would make the system easier to extend without changing its core inference logic.

F. Known Interface Inconsistency

The source code also defines three motive widgets. However, those motive values never enter the evidence dictionary used in the Bayesian query. Consequently, the app gives the appearance of considering motive while actually ignoring it. This is not a minor cosmetic issue. It affects interpretability because a user may reasonably believe motive contributes to the posterior even though it does not.

In addition, the same Python variable name is reused for two of the motive controls, meaning one of the interface values overwrites the other. Because the motive variables are unused in inference, the overwrite does not change the final probabilities, but it remains a clear code quality issue.

VII. WORKED INFERENCE EXAMPLE

To make the model more concrete, consider a scenario intended to favor the cat burglar:

$$FE = Yes, \quad AC = No, \quad FP = C, \quad SF = No.$$

Using the CPDs from the source code, the unnormalized posterior terms are:

$$\begin{aligned} \tilde{P}(A) &= \frac{1}{3}(0.1)(0.3)(0.05)(0.6), \\ \tilde{P}(B) &= \frac{1}{3}(0.1)(0.3)(0.05)(0.6), \\ \tilde{P}(C) &= \frac{1}{3}(0.9)(0.8)(0.2)(0.8). \end{aligned} \tag{7}$$

After normalization, the resulting posterior is approximately:

$$P(A | e) = 0.29\%, \quad P(B | e) = 0.29\%, \quad P(C | e) = 99.42\%. \tag{8}$$

This example illustrates three important points. First, the network behaves consistently with its encoded narrative assumptions. Second, the exact reasoning process is transparent enough to compute manually. Third, the confidence of the posterior is entirely a product of the chosen CPDs, not of learned evidence.

VIII. REPRESENTATIVE RESULTS AND BEHAVIORAL ANALYSIS

Several representative evidence combinations were derived directly from the encoded CPDs and are shown in Table VI. These examples are not benchmark tests in the statistical sense because the project has no dataset; rather, they are analytical sanity checks demonstrating how the model behaves under coherent evidence patterns.

These outputs reveal that the network is highly sensitive to agreement between multiple clue likelihoods. Such sensitivity is acceptable and even desirable in a didactic prototype because it visually demonstrates the compounding effect of evidence. However, a mature system would likely require calibration, uncertainty analysis, or at least a discussion of why those specific numerical values were chosen.

IX. VALIDATION STRATEGY

Because the project does not use an external dataset, evaluation cannot be performed through conventional train-test accuracy. Instead, validation should focus on internal consistency, correctness of probability calculations, and whether representative evidence patterns produce logically expected results. For this type of educational Bayesian application, the most important validation question is not “does the model match real crime data?” but rather “does the implemented system behave consistently with the assumptions encoded in the CPDs?”

Three validation layers are appropriate. First, the model should pass structural and CPD validation using `model.check_model()`, which is already present in the implementation. Second, posterior outputs should be checked to ensure that all probabilities are non-negative and sum to one. Third, scenario-based tests should confirm that strong evidence for a suspect increases that suspect’s posterior probability. These checks are summarized in Table VII.

A minimal automated test suite could be written without testing Streamlit directly. For example, tests could import `build_bayesian_network()`, run a small set of inference queries, and verify that the highest-ranked suspect matches the intended evidence pattern. This would make the project more robust while keeping the testing burden reasonable. It would also help future maintainers safely modify CPDs or add new evidence variables.

X. DISCUSSION

The project demonstrates an important tradeoff between interpretability and realism. The current network is highly interpretable because every evidence variable has one direct parent. A user can easily understand the meaning of the graph: the culprit influences the clues. This clarity is one of the strongest features of the project. In contrast, a more realistic model would likely include additional dependencies, hidden variables, and uncertainty about clue reliability. Such a model might be more expressive, but it would also be harder to explain in a short classroom demonstration.

TABLE VI
REPRESENTATIVE POSTERIOR OUTCOMES DERIVED FROM THE IMPLEMENTED MODEL

Scenario	Evidence combination	P(A)	P(B)	P(C)	Interpretation
No evidence	None	33.33%	33.33%	33.33%	Uniform prior, no suspect favored
Scholar-favoring	$FE = No, A_A = No, FP = A, SF = Yes$	96.77%	2.76%	0.46%	Scholar becomes dominant due to missing alibi and matching prints
Butler-favoring	$FE = No, A_B = No, FP = B, SF = Yes$	2.76%	96.77%	0.46%	Butler becomes dominant under analogous evidence
Cat-burglar-favoring	$FE = Yes, A_C = No, FP = C, SF = No$	0.29%	0.29%	99.42%	Strong convergence to the cat burglar profile

TABLE VII
RECOMMENDED VALIDATION CHECKS

Validation type	Example check	Reason
Model validity	<code>check_model()</code> returns true	Confirms graph and CPDs are compatible
Probability sanity	Posterior sums to 1	Ensures inference result is normalized
Scenario behavior	Fingerprint match raises suspect probability	Tests expected model response
Interface consistency	UI controls affect evidence dictionary	Prevents misleading inputs
Environment check	Dependencies install from requirements	Supports reproducibility

Another useful aspect of the project is that it exposes how numerical assumptions shape AI output. For instance, if the probability of forced entry given the cat burglar were reduced from 0.9 to 0.6, the same evidence would produce less extreme posterior confidence. This means the application can be used not only to demonstrate Bayesian inference, but also to discuss model sensitivity. In AI education, this is valuable because students often see final predictions without seeing the assumptions that produced them.

The project also shows why interface design matters in AI systems. A probabilistic model may be mathematically correct, but if the interface displays controls that are not actually connected to inference, users can form the wrong mental model of the system. The unused motive controls are therefore a meaningful design issue. They provide a concrete example of the gap that can appear between a model’s internal reasoning and a user’s understanding of that reasoning.

XI. REPRODUCIBILITY AND ENVIRONMENT ANALYSIS

A. Observed Environment Problems

One of the most significant practical issues in the downloaded artifact is the bundled `venv/` directory. The virtual environment refers to a Python executable path that exists on a different user’s machine. This makes the environment non-portable. When the included interpreter is invoked on the current system, it fails because the referenced executable path

is missing. This is a classic reproducibility issue in student projects shared through cloud storage.

B. Current Machine Status

Inspection of the present machine shows that:

- Python 3.10.11 is installed globally.
- `pandas` is installed globally.
- `streamlit` is not installed globally.
- `pgmpy` is not installed globally.
- The Graphviz executable `dot` is not available in the system path.

As a consequence, the project does not run immediately in its downloaded form even though the source code itself is valid enough to analyze.

C. Recommended Reproduction Procedure

The proper solution is to rebuild the environment locally from the dependency list:

```
cd I:\Mystery_Solver\Mystery_Solver
py -3.10 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m streamlit run main.py
```

For complete visualization support, Graphviz should also be installed separately at the operating-system level [6]. Without that installation, the inference engine will still work, but the structural graph may not render correctly.

XII. CRITICAL EVALUATION

A. Strengths

The strongest aspects of the project are conceptual rather than infrastructural:

- The project successfully demonstrates a major AI concept in a concrete and understandable way.
- The Bayesian formulation is mathematically coherent and validated through `check_model()`.
- The interface is lightweight, accessible, and suitable for rapid experimentation.
- The output is interpretable because the system returns full probability distributions rather than hidden decisions.
- The codebase is small enough to be read and understood end-to-end.

TABLE VIII
KEY WEAKNESSES IDENTIFIED DURING SOURCE INSPECTION

Issue	Severity	Impact
Unused motive inputs	High	Misleads the user about what evidence affects inference
Shared broken virtual environment	High	Prevents immediate reproducibility on another machine
No automated tests	Medium	Makes regression and validation harder
Unused dependencies	Low/Medium	Creates environment noise and weakens packaging quality
Encoding problem in title text	Low	Reduces polish in the interface
No README or design note	Medium	Increases onboarding difficulty for evaluators and users

B. Weaknesses

The most important weaknesses are summarized in Table VIII.

C. Threats to Validity

There are also validity limitations specific to the model design:

- **Subjective parameters:** the CPDs are manually chosen and not estimated from data.
- **Simplified dependencies:** all clues are modeled as conditionally independent given guilt.
- **No uncertainty over model structure:** the graph itself is treated as fixed and unquestioned.
- **No empirical validation:** there is no held-out dataset or synthetic benchmark suite.

These are not fatal flaws for a course demonstration, but they should be acknowledged explicitly in a professional report.

D. Ethical and Pedagogical Reflection

Even though the project uses a fictional setting, it still raises a useful AI lesson: probabilistic outputs can look authoritative even when they are generated from subjective assumptions. In classroom use, that is not a drawback if it is explained honestly. In fact, this can be turned into a teaching point about responsible AI: a cleanly presented probability distribution does not automatically mean the underlying model is objective, fair, or empirically justified.

XIII. IMPROVEMENT ROADMAP

If the project were to be extended beyond its current form, the following improvements would provide the highest value:

A. Model-Level Improvements

- Add explicit motive nodes such as `MotiveA`, `MotiveB`, and `MotiveC`.
- Introduce more realistic dependencies, for example between forced entry and security evidence.
- Consider modeling reliability or uncertainty of each clue source.
- Add scenario presets and counterfactual queries to improve interpretability.

B. Software-Level Improvements

- Add a proper `README.md` documenting purpose, installation, and assumptions.
- Rename `requirements.txt.txt` to `requirements.txt`.
- Replace the copied `venv/` with a recreated environment and a `.gitignore`.
- Add tests for model creation, posterior normalization, and selected evidence cases.
- Fix the motive-widget overwrite and title encoding issues in the user interface.

C. Report-Level Improvements

For academic submission quality, the project would also benefit from:

- screenshots of the running interface,
- explicit pseudocode for the inference flow,
- comparison against an alternative baseline such as naive scoring rules,
- and a short user study or classroom demo reflection.

XIV. FINAL DELIVERABLES AND ACADEMIC CONTRIBUTION

The final project deliverable is more than a small Streamlit script. It is a complete demonstration of how an artificial intelligence concept can be transformed into a usable interactive artifact. The project begins from a reasoning problem, expresses that problem as a probabilistic graphical model, implements the model in software, and presents the result through a user-facing interface. This makes the work suitable for an artificial intelligence course because it combines theory, implementation, and interpretation.

The major deliverables are summarized in Table IX. These deliverables show that the project contains both a technical component and an explanatory component. The technical component is the application itself: it builds a Bayesian Network, accepts evidence, runs inference, and displays probabilities. The explanatory component is equally important: the project must make clear what assumptions are encoded in the model and what limitations follow from those assumptions.

The academic contribution of the project is its clarity. A more complex project might use larger datasets or deeper algorithms, but it could also hide the reasoning process from the user. In contrast, `Mystery_Solver` keeps the model small enough that every probability shift can be traced back to a CPD. This is valuable in an AI course because it supports explainability. The user can see that the system is not guessing randomly; it is applying a structured set of assumptions.

The project also encourages critical thinking about AI outputs. The posterior probabilities are precise numerical values, but they are not objective facts. They depend on prior probabilities, conditional probabilities, and graph structure. Therefore, the application teaches both the power and the limitation of probabilistic AI: it can combine evidence coherently, but the quality of its conclusions depends on the quality of the model design.

TABLE IX
FINAL PROJECT DELIVERABLES

Deliverable	Status	Academic value
Bayesian model	Completed prototype	Demonstrates probabilistic representation and CPD design
Inference engine	Completed prototype	Shows exact posterior computation through variable elimination
Interactive interface	Completed prototype	Allows users to experiment with evidence combinations
Visualization support	Partial	Helps explain network structure, but depends on Graphviz setup
Documentation/report	Completed in report form	Explains design, assumptions, limitations, and future work
Testing infrastructure	Recommended future work	Would improve reproducibility and confidence in changes

XV. CONCLUSION

This report has presented a full technical analysis of `Mystery_Solver`, a compact AI application that implements probabilistic reasoning through a Bayesian Network and exposes the result in an interactive Streamlit interface. The project succeeds as a didactic demonstration because it makes the relationship between evidence, probability, and inference highly visible. The Bayesian model is simple, interpretable, and mathematically valid for its intended use. The code structure is also reasonably understandable, with a beneficial separation between interface and model logic.

At the same time, the project clearly remains a prototype rather than a polished final software product. The most important shortcomings are engineering-related: environment portability problems, unused interface inputs, absent documentation, and a lack of tests. These issues do not erase the

conceptual strengths of the project, but they do reduce its quality as a reproducible academic artifact.

Overall, `Mystery_Solver` demonstrates that even a small codebase can meaningfully embody core ideas from artificial intelligence. It offers a strong foundation for explaining Bayesian reasoning, and with moderate revision it could evolve into a substantially stronger course submission or showcase project.

ACKNOWLEDGMENT

This report was written specifically for the submitted project and was based on direct inspection of the provided source code to avoid plagiarism. An AI writing assistant, ChatGPT by OpenAI, was used to help organize and refine the report prose; project-specific technical claims were then checked against the source files and official documentation [8].

REFERENCES

- [1] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.
- [2] N. L. Zhang and D. Poole, "Exploiting causal independence in Bayesian network inference," *Journal of Artificial Intelligence Research*, vol. 5, pp. 301–328, 1996, doi: 10.1613/jair.305.
- [3] A. Ankan and A. Panda, "pgmpy: Probabilistic Graphical Models using Python," in *Proceedings of the 14th Python in Science Conference*, 2015, pp. 6–11, doi: 10.25080/Majora-7b98e3ed-001.
- [4] "pgmpy Documentation," pgmpy. [Online]. Available: <https://pgmpy.org/>. Accessed: Apr. 28, 2026.
- [5] "Streamlit Documentation," Streamlit. [Online]. Available: <https://docs.streamlit.io/>. Accessed: Apr. 28, 2026.
- [6] "Graphviz Documentation," Graphviz. [Online]. Available: <https://graphviz.org/documentation/>. Accessed: Apr. 28, 2026.
- [7] "Python Documentation," Python Software Foundation. [Online]. Available: <https://docs.python.org/>. Accessed: Apr. 28, 2026.
- [8] "ChatGPT Overview," OpenAI. [Online]. Available: <https://openai.com/chatgpt/overview/>. Accessed: Apr. 28, 2026.